KUBERNETES ON AWS USING EKS

1. Create Kubernetes Cluster using EKS (Elastic Kubernetes Service)

- **What is EKS?**
- Amazon EKS (Elastic Kubernetes Service) is a managed Kubernetes service that simplifies deploying, managing, and scaling containerized applications using Kubernetes on AWS.

- ***Steps to Create an EKS Cluster***
- **Pre-requisites:**
- AWS CLI installed and configured
- eksctl installed
- IAM permissions to create VPC, EKS cluster, and worker nodes

2. Create EKS Cluster:

- eksctl create cluster \--name demo-cluster \--region us-west-2 \--nodegroup-name demo-nodes \--node-type t3.medium \ --nodes 2 \--nodes-min 1 \--nodes-max 3 \--managed

3. Install kubectl and Access EKS Cluster

Install kubectl:

- curl -o kubectl https://amazon-eks.s3.us-west-2.amazonaws.com/latest/bin/linux/amd64/kubectl
- chmod +x ./kubectl
- sudo mv ./kubectl /usr/local/bin

4. Update Kubeconfig to Access EKS:

- *aws eks update-kubeconfig --region us-east-2 --name demo-cluster*

5. Verify Cluster Connection

- *kubectl get svc*

- **Introduction to Pods and Services**
- ***Pod:***
  - *The smallest and simplest Kubernetes object.*
  - *Represents a single instance of a running process.*
  - *Can contain one or more containers*
- ***Service:***
  - *An abstraction that exposes a set of pods as a network service.*

Types:

- *ClusterIP (default)*
- *NodePort*
- *LoadBalancer*

- **Main Container and Sidecar Containers**
  - ***Main Container:*** *The primary workload in the pod (e.g., your application).*
  - ***Sidecar Container:*** *Runs alongside the main container to provide supporting functionality.* **Examples**: *logging agent, monitoring agent, proxy*

1. **Run First Pod using kubectl run**
   - *kubectl run nginx-pod --image=nginx*
   - *kubectl get pods*
2. ***Verify Pod Logs:***
   - *kubectl logs nginx-pod*
3. **Expose Pod using kubectl expose**
   - *kubectl expose pod nginx-pod –type NodePort --port=80*
4. ***Check Service:***
   - *kubectl get svc*
5. ***Access Application (if on AWS EC2):***

- *http://<publicIP>:<NodePort*

[GENERAL INTERVIEW QUESTIONS]

Q1. What are the main components created by eksctl during cluster creation?

- eksctl creates the EKS cluster, worker node group (EC2 instances), IAM roles, security groups, and the VPC network needed for the cluster.

Q2. How do you configure kubectl to access an EKS cluster?

- ***Use the command:->*** aws eks --region update-kubeconfig –name

Q3. What is the purpose of the aws eks update-kubeconfig command

- It updates your kubeconfig file with EKS cluster credentials and endpoint information, allowing kubectl to interact with the cluster.

Q4. Why do we need Pods instead of directly running containers

- Pods are needed in Kubernetes **because** they allow grouping of **one** or **more tightly** coupled containers that share the same network, storage, and lifecycle.
- Kubernetes **manages Pods** as the smallest unit of deployment, making orchestration, scheduling, and scaling more effective and flexible than managing standalone containers.

Q5. What's the difference between a Main container and a Sidecar container in the same Pod?

- The ***Main container runs*** the core application logic, while the **Sidecar supports** it (e.g., managing logs, handling network traffic, or metrics collection)

Q6. How do containers within a Pod communicate with each other?

- Containers in the same Pod share the same localhost (network namespace) and can ***communicate*** over localhost using ports

Q7. What are the types of Services in Kubernetes?

- **ClusterIP** (**default**) **–** Internal only
- **NodePort** – Exposes on a static port on each node
- **LoadBalancer –** Exposes externally via cloud provider's load balancer
- **ExternalName –** Maps to external DNS

Q8. How do you verify that a Pod is running successfully?

- kubectl get pods
- kubectl describe pod
- kubectl logs

Q9. What happens when a Pod dies in Kubernetes?

- **Pod fails (crash, node failure, etc.)**
- **Controller detects** that the Pod is no longer running.
- Controller creates a **new Pod (with a new UID and IP)** to replace the failed one.
- Service automatically **routes traffic** to the new Pod.